

TMO – Temporal Multi-Omics

Complete Methods (Final Pipeline)

Phabel A. López-Delgado

May 2026

1 Data Preprocessing and Gene-Level ATAC Matrix

Input: 10x Multiome dataset consisting of

- `filtered_feature_bc_matrix.h5`: paired RNA and ATAC peak counts.
- `atac_peak_annotation.tsv`: mapping from ATAC peaks to proximal genes.

Procedure:

1. Read the H5 file using `muon.read_10x_h5()` to obtain a MuData object with `rna` and `atac` modalities.
2. Build a binary peak-gene incidence matrix $M \in \{0, 1\}^{P \times G}$, where P is the number of peaks and G the number of unique genes. $M_{p,g} = 1$ if peak p is annotated to gene g (from the TSV).
3. Compute the gene-level ATAC matrix (cells $\times$ genes):

$$A_{\text{gene}} = X_{\text{peak}} \cdot M,$$

where X_{peak} is the sparse peak-by-cell count matrix. A_{gene} is stored in `.obsm['ATAC_gene']` as a CSR matrix.

4. RNA counts remain in `.X` and are normalised later.

2 Pseudotime from First Diffusion Component

Goal: Assign a continuous pseudotime $\tau \in [0, 1]$ to each cell reflecting developmental progression, without requiring a known root cell.

Steps:

1. Normalise RNA counts to 10 000 total counts per cell, apply $\log(1 + x)$.
2. Select top 2000 highly variable genes (Seurat flavour).
3. PCA (30 components), k -nearest-neighbour graph ($k = 15$), diffusion maps (15 components).
4. Take the first diffusion component as initial pseudotime τ_0 .
5. Min-max scale to $[0, 1]$.

Optional reversal with an early marker gene:

- If a marker gene is provided (e.g., *Pax6*, *CD34*), compute Pearson correlation r between its expression and τ_0 .

- If $r < 0$, set $\tau = 1 - \tau_0$ and store flag ‘pseudotime_reversed’ = True.
- Otherwise keep $\tau = \tau_0$, flag = False.

The flag is saved in `.uns['pseudotime_reversed']`. During evaluation and plotting, $\Delta\tau$ signs are flipped if the flag is True, ensuring that positive $\Delta\tau$ always means ATAC leads RNA (priming).

3 Latent Representations

To reduce dimensionality:

- **RNA latent:** PCA with 50 components on the log-normalised RNA matrix, yielding $Z_{\text{RNA}} \in \mathbb{R}^{N \times 50}$.
- **ATAC latent:** First apply TF-IDF transformation to A_{gene} :

$$X_{\text{tfidf}} = A_{\text{gene}} \cdot \log(N/df),$$

normalised to ℓ_2 norm per cell. Then truncated SVD with 50 components: $Z_{\text{ATAC}} \in \mathbb{R}^{N \times 50}$.

These 100 tokens (50 ATAC + 50 RNA) per cell are the input to TMO.

4 Local Cross-Correlation Function (CCF) for Ground-Truth $\Delta\tau$

For each matched component pair (i, i) , we compute a signed, time-resolved cross-correlation along pseudotime.

Let $a_i(\tau)$ and $r_i(\tau)$ be the standardised ATAC and RNA component scores. Slide a window of half-width $w = 0.1$ across pseudotime at centres $\tau_0 \in \{0.1, 0.2, \dots, 0.9\}$. Inside each window I :

$$\text{CCF}_i(\tau_0, \ell) = \frac{1}{|I|} \sum_{t \in I} a_i(\tau_t) \cdot r_i(\tau_t + \ell),$$

where ℓ is discretised over $[-0.3, 0.3]$ (61 points) and $\tau_t + \ell$ is linearly interpolated using `ccf_single_gene` (interpolation onto a regular grid). The peak lag is:

$$\Delta\tau_i(\tau_0) = \arg \max_{\ell} \text{CCF}_i(\tau_0, \ell).$$

The surface $\Delta\tau_i(\tau)$ is smoothed with Gaussian process regression (Matern-3/2 kernel, bounds: `length_scale` $[10^{-2}, 0.5]$, `noise_level` $[10^{-4}, 10^0]$) to produce $\Delta\tau_i^{\text{smooth}}(\tau)$. The training target is the smoothed value at the last window ($\tau = 0.9$): $\Delta\tau_i^{\text{target}}$.

If `.uns['pseudotime_reversed']` is True, all $\Delta\tau_i^{\text{target}}$ are multiplied by -1 .

5 TMO Model Architecture

Notation

- N : batch size
- $d_{\text{model}} = 64$
- $n_{\text{heads}} = 4$
- $n_{\text{layers}} = 2$

- $n_{\text{comps}} = 50$
- $\Delta\tau_{\text{max}} = 0.5$
- $\sigma_{\text{min}}^2 = 0.001$

Tokenisation

Each cell is represented by 100 tokens (50 ATAC, then 50 RNA). The embedding for a token is:

$$x = \text{proj}(\text{latent_value}) + \text{modality_embedding}(m) + \text{pseudotime_embedding}(\tau),$$

where:

- proj is a linear layer ($1 \rightarrow d_{\text{model}}$) applied to the scalar PCA/LSI value. This gives each token cell-state-specific information.
- $\text{modality_embedding}$: learnable embedding, 0 = RNA, 1 = ATAC.
- $\text{pseudotime_embedding}$: 2-layer MLP ($128 \rightarrow 64 \rightarrow d_{\text{model}}$) mapping τ .

Two-Pass Forward

Pass 1 (unbiased). Tokens $\rightarrow$ transformer encoder with no cross-attention bias. After encoding, mean-pool the ATAC token outputs $\rightarrow$ cell embedding $z_c \in \mathbb{R}^{d_{\text{model}}}$.

Lag and width prediction. For each ATAC component i and each RNA component j , a shared LagMLP / WidthMLP takes as input $[z_c; e_{\text{comp}}]$ (where e_{comp} is a learnable component embedding) and predicts a signed lag $\hat{\Delta}\tau$ and a window width $\hat{\sigma}^2$.

LagMLP architecture:

$$\begin{aligned} h &= \text{LayerNorm}(\text{concat}(z_c, e_{\text{comp}})) \\ h &= \text{GELU}(\text{Linear}(2d_{\text{model}} \rightarrow d_{\text{model}})) \\ h &= \text{Dropout}(0.1) \\ h &= \text{GELU}(\text{Linear}(d_{\text{model}} \rightarrow d_{\text{model}}/2)) \\ h &= \text{Dropout}(0.1) \\ \hat{\Delta}\tau_{\text{raw}} &= \text{Linear}(d_{\text{model}}/2 \rightarrow 1) \\ \hat{\Delta}\tau &= \Delta\tau_{\text{max}} \cdot \tanh(\hat{\Delta}\tau_{\text{raw}}) \end{aligned}$$

WidthMLP: identical structure, output passed through Softplus plus a minimum $\sigma_{\text{min}}^2 = 0.001$:

$$\hat{\sigma}^2 = \sigma_{\text{min}}^2 + \text{Softplus}(\text{MLP}_{\text{Width}}(z_c, e_{\text{comp}})).$$

For each cell we obtain:

$$\hat{\Delta}\tau_{\text{ATAC}}, \hat{\sigma}_{\text{ATAC}}^2 \in \mathbb{R}^{N \times 50}, \quad \hat{\Delta}\tau_{\text{RNA}}, \hat{\sigma}_{\text{RNA}}^2 \in \mathbb{R}^{N \times 50}.$$

Pass 2 (biased). The same tokens are processed again. Cross-attention biases are now active. For an ATAC token a_i attending to an RNA token r_j :

$$b_{ij} = - \frac{(\tau_j - \tau_i - \hat{\Delta}\tau_{\text{RNA},j})^2}{2\hat{\sigma}_{\text{RNA},j}^2}.$$

For the reverse direction (RNA $\rightarrow$ ATAC):

$$b_{ji}^{\text{rev}} = - \frac{(\tau_i - \tau_j + \hat{\Delta}\tau_{\text{ATAC},i})^2}{2\hat{\sigma}_{\text{ATAC},i}^2}.$$

Biases are added to the attention logits before softmax.

Decoder. After Pass 2, the RNA token outputs are linearly projected to reconstruct the RNA latent components $\hat{R} \in \mathbb{R}^{N \times 50}$.

Symmetric Baseline

The same model is used but with `use_bias=False`, so only Pass 1 runs and no lag loss is applied.

6 Training Loss Function

The asymmetric model is trained on the full dataset with:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{recon}} \cdot \mathcal{L}_{\text{recon}} + \lambda_{\text{lag}} \cdot \mathcal{L}_{\text{lag}},$$

where $\lambda_{\text{recon}} = 1.0$, $\lambda_{\text{lag}} = 0.1$.

$$\mathcal{L}_{\text{recon}} = \text{MSE}(\hat{R}, R_{\text{target}}) \quad (\text{reconstruction of RNA components}).$$

For each ATAC component i , compute the mean predicted lag over cells:

$$\bar{\Delta}\tau_i = \frac{1}{N} \sum_c \hat{\Delta}\tau_{\text{ATAC}}(c, i)$$

Then

$$\mathcal{L}_{\text{lag}} = \text{MSE}(\{\bar{\Delta}\tau_i\}, \{\Delta\tau_i^{\text{target}}\}).$$

The symmetric baseline uses only $\mathcal{L}_{\text{recon}}$ (no lag loss).

Optimiser: Adam ($\text{lr} = 10^{-3}$), StepLR scheduler (`step_size = 20`, $\gamma = 0.5$). Training runs for 50 epochs. After every 5 epochs, the Lag Concordance Score (LCS) – Spearman correlation between predicted per-component lags and the CCF targets – is computed on the full dataset (in-set). The model with the highest LCS is saved as `tmo_asymmetric_best.pt`.

7 Validation Metrics

Lag Concordance Score (LCS)

For each ATAC component i , the mean predicted lag $\bar{\Delta}\tau_i$ is compared to the CCF-derived target $\Delta\tau_i^{\text{target}}$. $\text{LCS} = \text{Spearman } \rho(\{\bar{\Delta}\tau_i\}, \{\Delta\tau_i^{\text{target}}\})$. Reported on the full dataset; values > 0.9 indicate the model captures the temporal ordering of regulatory lags.

ChIP-seq Validation (deterministic, full background)

For a transcription factor, its top-500 bound genes are obtained from ChIP-Atlas (peak-to-gene mapping ± 5 kb from TSS). Component-level CCF lags are projected to gene-level lags via the PCA loadings:

$$\text{gene_lag} = (\text{normalised_loadings})^\top \cdot \text{comp_lag_ccf}.$$

The background set is defined as **all non-target genes** (optionally restricted to those within ± 0.5 log-expression units of the target-gene median to control for expression level). No random

subsampling is performed; the full set of eligible background genes is used, making the test fully deterministic. A two-sided Mann-Whitney U test compares the $\Delta\tau$ values of target genes against this background. This validation does not rely on any model predictions — it uses only the data-derived CCF surface — and therefore provides an independent, orthogonal assessment of whether the regulatory lag signal is associated with physical transcription factor binding.

Perturb-seq Validation ($\delta\Delta\tau$)

For a knockout (e.g., SMARCB1), the asymmetric model is trained on control cells. $\delta\Delta\tau = |\Delta\tau_{\text{perturbed}} - \Delta\tau_{\text{control}}|$ is computed for each ATAC component. Target-associated components are compared against background using a one-sided Mann-Whitney U test.

8 Uniform Biological Interpretation of $\Delta\tau$

After pseudotime reversal is accounted for:

- Positive $\Delta\tau$ (red in heatmaps) $\rightarrow$ ATAC leads RNA (chromatin priming).
- Negative $\Delta\tau$ (blue) $\rightarrow$ RNA leads ATAC (activity-dependent regulation).

The correlation heatmap (Figure 2) uses absolute Pearson correlation (always ≥ 0) and reveals a U-shape characteristic of developmental decoupling.

9 Dependencies and Reproducibility

Python 3.10, PyTorch 2.1, scanpy 1.9, muon 0.6, scikit-learn 1.3, scipy, matplotlib, seaborn, pandas, numpy, tqdm, gseapy 1.0.

The complete pipeline is provided as numbered scripts, and the core library `tmo` is open-source under the MIT license.